

DSA -- Heap & Hashing

[Target] Target: Explain Max Heap vs Min Heap in 20s. Name 2 collision resolution techniques in 10s.
Read time: 12 minutes

Heap Data Structure

A Heap is a special Tree-based data structure in which the tree is a Complete Binary Tree.

Every node's value follows a specific order rule relative to its children -- always a complete binary tree.

```

    ...
      A      array index 1
     /\
    B  C     index 2, 3
   /\ /\
  D E F G   index 4, 5, 6, 7
Array representation: [A, B, C, D, E, F, G]
Positions:           [1, 2, 3, 4, 5, 6, 7]
    ...
```

Heap Array Index Formula

If a node is at index i (1-indexed):

Left child index $2*i$
Right child index $2*i + 1$
Parent index $i/2$
Example: Node at index 3
- Left child = $2 * 3 =$ index 6
- Right child = $2 * 3 + 1 =$ index 7
- Parent = $3 / 2 =$ index 1

This formula assumes heap starts at index 1, not 0.
Height of a complete binary tree of n nodes = $\lceil \log_2 n \rceil$

Max Heap vs Min Heap

Feature

Max Heap
Min Heap

Rule

Parent $\geq$ both children
Parent $\leq$ both children

Root value

Maximum element
Minimum element

Use case

Find max quickly, Heap Sort
Priority Queue (min first), Dijkstra

Max Heap Example:

```

50      root = maximum
  /\
 30 20
 /\ /\
15 10 8 16
Verify: 50 > 30, 20 [OK] | 30 > 15, 10 [OK] | 20 > 8, 16 [OK]
```

Min Heap Example:

```

10      root = minimum
  /\
 30 20
```

/ \ / \
35 40 32 25
Verify: 10 < 30, 20 [OK] | 30 < 35, 40 [OK] | 20 < 32, 25 [OK]

Heap Operations

Operation
Time Complexity
What it does

Heapify
 $O(n)$
Build a heap from an unsorted array

Insertion
 $O(\log n)$
Add element, bubble up to restore heap property

Deletion
 $O(\log n)$
Remove root, replace with last, bubble down

Peek
 $O(1)$
Read root (max or min) without removing

Heapify = process of creating a heap from an array (not inserting one-by-one)

Your 20-Second Script -- Heap

"A heap is a complete binary tree satisfying the heap property. In a max heap, every parent is greater than or equal to its children -- so the root always holds the maximum. In a min heap, every parent is less than or equal to its children -- root holds the minimum. Insertion and deletion are $O(\log n)$. It's the preferred data structure for Priority Queues."

B-Tree vs B+Tree

These are two variants of the B-Tree used in databases. Key difference is where data is stored.

Feature
B-Tree
B+Tree

Data stored in
Leaf AND internal nodes
Leaf nodes only

Internal nodes contain
Keys + Data pointers
Keys only (no data)

Leaf nodes linked?
Not linked
[OK] Linked like a linked list

Search
Slower (data may be anywhere)
Faster (always reach leaf)

Deletion
Complex
Easy (always from leaf)

Range queries
Less efficient
Very efficient (follow leaf links)

Use in
General indexes
Most modern DBs (MySQL InnoDB)

B-Tree:		B+Tree:
[key data] internal		[key only] internal
/ \		/ \
[key data] [key data]		[key data] [key data] [key data]
		leaf nodes linked together

Interview line: "Most modern databases use B+Tree because leaf nodes are linked -- making range queries like BETWEEN fast and predictable."

Hashing
Hashing = mapping keys and values into a hash table using a hash function for $O(1)$ average access.
Key Hash Function Index in Hash Table Value

"Hashing stores and retrieves data in $O(1)$ time."

Hash Function Types

Function
Formula
Example (key=24, table size=10)

Division Method
 $K \bmod n$
 $24 \bmod 10 = 4$

Mid Square
Square key, take middle digits
 $24^2 = 576$ middle = 7

Folding Method
Split key into parts, add them
--

Worked Example (from ebook)
Keys: 24, 52, 91, 67, 48, 83 Hash Function: $K \bmod 10$

Key
 $K \bmod 10$
Stored at index

24
4
4

52
2
2

91
1
1

67
7
7

48
8
8

83
3
3

Collision in Hashing

A collision occurs when two different keys map to the same index in the hash table.

key1 hash(key1) = 5

key2 hash(key2) = 5 COLLISION

Collision Resolution Techniques

...

Collision Resolution

Chaining Open Addressing
(Open Hash) (Closed Hash)

Linear Quadratic Double Hashing
 Probing Probing

...

1. Chaining (Open Hashing)

Each table index holds a linked list

Colliding keys go into the same list

No limit on number of keys per bucket

Index 5 [key1] [key5] NULL

[OK] Simple | [OK] No table overflow | Extra memory for pointers

2. Open Addressing (Closed Hashing)

All keys stored inside the table itself -- no external lists

On collision, probe for the next available slot

Technique

Next slot formula

Problem

Linear Probing

$(h(k) + i) \bmod n$

Clustering -- keys bunch together

Quadratic Probing
 $(h(k) + i^2) \bmod n$
Secondary clustering

Double Hashing
 $(h_1(k) + i \cdot h_2(k)) \bmod n$
Best distribution, no clustering

Interview line: "Chaining handles collision with linked lists -- no size limit. Open Addressing keeps all keys in the table and probes for the next empty slot. Double hashing is the best open addressing technique."

Your 40-Second Script -- Hashing

"Hashing maps keys to indices in a hash table using a hash function -- this gives $O(1)$ average access time. The most common hash function is division: $\text{key} \bmod \text{table size}$. When two keys map to the same index, that's a collision. Two main resolution strategies: Chaining uses a linked list at each index, Open Addressing probes for the next empty slot using Linear, Quadratic, or Double Hashing."

Follow-Up Questions (Expect These)
Q: What is the load factor in hashing?

Load factor = (number of elements) / (table size). High load factor → more collisions → performance degrades. Rehashing is done when load factor exceeds a threshold (typically 0.7-0.75).

Q: What is the worst case for a hash table?

$O(n)$ -- when all keys hash to the same index, the linked list (chaining) or probe sequence degrades to linear search.

Q: Why is Double Hashing better than Linear Probing?

Linear Probing causes primary clustering -- consecutive filled slots slow down subsequent insertions. Double Hashing uses a second hash function to determine the probe step, distributing keys more uniformly.

Q: B-Tree vs B+Tree -- which is used in MySQL?

MySQL InnoDB uses B+Tree for indexes. Leaf nodes are linked, enabling fast range queries.